Principles of Cryptocurrency Design
Appunti ed Esercizi

Ale Sandu

Uno dei vari aspetti critici dei sistemi di criptovalute è la necessità di raggiungere un "consenso", da parte di agenti dislocati in posizioni diverse, sulla cosiddetta blockchain.

1 Il problema Byzantine Broadcast

Si consideri il seguente sistema distribuito:

- Un insieme di n nodi, $[n] = \{1, 2, \ldots, n\}$;
- Ogni nodo può eseguire computazioni locali arbitrarie e inviare messaggi di lunghezza arbitraria a ogni altro nodo.

Il problema computazionale di cui ci occupiamo è chiamato *Byzantine Broadcast* (BB). Degli n nodi, ce ne sono f corrotti, con $0 \leq f \leq n$; gli altri $n - f$ nodi sono onesti. Al nodo 1 (la sorgente) viene affidato in input un messaggio b. Vogliamo progettare un protocollo al termine del quale ogni nodo onesto i dia in output un valore y_i tale che:

- **Consistency:** se i e j sono due nodi onesti, allora $y_i = y_j$;
- **Validity:** se la sorgente è un nodo onesto, allora $y_i = b$ (il messaggio affidato in input alla sorgente) per ogni nodo onesto i.

Gli $n - f$ nodi onesti applicheranno il nostro protocollo. Gli f nodi corrotti non sappiamo come si comporteranno: la nostra assunzione *worst-case* è che tutti i nodi corrotti si coalizzeranno per far fallire il nostro protocollo. Non sappiamo chi sono i nodi corrotti.

2 Modelli di calcolo distribuito e assunzioni

Il modo in cui possiamo progettare e analizzare rigorosamente protocolli distribuiti per un problema come il Byzantine Broadcast dipende drasticamente dal modello che usiamo, ossia da ciò su cui assumiamo di poter o non poter contare. In particolare, qui ci concentreremo su queste tre questioni:

1. **Synchronous / Asynchronous.** Quando un nodo invia un messaggio, possiamo assumere che il messaggio arrivi a destinazione? Se sì, possiamo assumere un upper bound al tempo che impiega un messaggio per arrivare a destinazione? A seconda che la risposta a questa domanda sia affermativa o negativa diremo che ci troviamo in un modello sincrono o asincrono. In letteratura si trovano anche "gradazioni intermedie" note come modelli parzialmente sincroni.
2. **Authenticated / Non authenticated.** Quando un nodo j riceve un messaggio proveniente da un nodo i, il nodo j può assumere che il mittente del messaggio sia effettivamente i oppure dobbiamo considerare la possibilità che qualche nodo corrotto possa far credere al nodo j di essere il nodo i quando questo non è vero? Quando un nodo j riceve un

messaggio da un nodo i e lo inoltra a un nodo k, il nodo k ha modo di verificare che il messaggio era stato effettivamente scritto da i? Vedremo che le risposte a queste domande distinguono se ci troviamo in un modello autenticato o non autenticato. In particolare la risposta alla seconda domanda è affermativa se c'è un *PKI (Public Key Infrastructure) setup*.

3. **Permissioned / Permissionless.** Gli agenti che partecipano al protocollo sono fissati dall'inizio? Oppure nuovi nodi possono aggiungersi durante l'esecuzione del protocollo? In tutti i risultati degli anni '80 si assume che gli n nodi siano fissati all'inizio. Oggi tipicamente chiamiamo un tale modello *permissioned*, per distinguerlo dal modello *permissionless* usato da Bitcoin e altre criptovalute, nel quale nuovi nodi possono aggiungersi in ogni momento durante l'esecuzione del protocollo.

3 Il modello sincrono, permissioned, con PKI setup

Per iniziare, mettiamoci nella configurazione più favorevole possibile. Assumiamo di essere in un sistema sincrono:

- C'è un *global clock* noto a tutti i nodi che scandisce il tempo in round discreti: $0, 1, 2, \ldots$;
- Ogni messaggio inviato in un round t arriva a destinazione prima dell'inizio del round $t+1$.

Assumiamo che ci sia un *Public Key Infrastructure (PKI) setup*:

- Ogni nodo i ha una coppia di chiavi (sk_i, pk_i);
- L'insieme delle chiavi pubbliche $\{pk_i : i \in [n]\}$ è *common knowledge* (noto a tutti) fra i nodi;
- Dato un messaggio m, indichiamo con $\langle m \rangle_i$ il messaggio m con l'aggiunta di una firma valida eseguita con la chiave segreta sk_i.

4 Il protocollo di Dolev–Strong

Consideriamo il protocollo di Dolev–Strong per il problema Byzantine Broadcast.

Algorithm 1 Dolev-Strong Protocol for Byzantine Broadcast

ROUND 0. Ogni nodo u inizializza un insieme vuoto $\mathcal{E}_u = \emptyset$.
    La sorgente (il nodo 1) riceve in input b e invia $\langle b \rangle_1$ a tutti i nodi.
PER OGNI ROUND $r = 1, \ldots, f$. Ogni nodo u:
    Per ogni messaggio $\langle \hat{b} \rangle_{1,\sigma_1,\sigma_2,\ldots,\sigma_{r-1}}$ con r firme distinte (inclusa quella
    della sorgente) ricevuto nel Round $r-1$:
        SE $\hat{b} \notin \mathcal{E}_u$:
            Aggiungi $\hat{b}$ a $\mathcal{E}_u$;
            Firma il messaggio $\langle \hat{b} \rangle_{1,\sigma_1,\sigma_2,\ldots,\sigma_{r-1}}$;
            Invia il messaggio firmato $\langle \hat{b} \rangle_{1,\sigma_1,\sigma_2,\ldots,\sigma_{r-1},\sigma_u}$ a tutti i nodi;
ROUND $f+1$. Ogni nodo u:
    Per ogni messaggio $\langle \hat{b} \rangle_{1,\sigma_1,\sigma_2,\ldots,\sigma_f}$ con $f+1$ firme distinte (inclusa quella
    della sorgente) ricevuto nel Round f:
        SE $\hat{b} \notin \mathcal{E}_u$, aggiungi $\hat{b}$ a $\mathcal{E}_u$;
    SE $\mathcal{E}_u$ contiene un solo elemento, allora OUTPUT l'elemento in $\mathcal{E}_u$,
    Altrimenti OUTPUT 0

Dimostrazione consistency.

Lemma 1. *Se un nodo onesto i ha un bit $\hat{b} \in \mathcal{E}_i$ alla fine di un round r, allora ogni nodo onesto j ha $\hat{b} \in \mathcal{E}_j$ alla fine del round $r+1$.*

dim. Lemma 1. Supponiamo che un nodo onesto i abbia $\hat{b} \in \mathcal{E}_i$ alla fine del round r. Sia $t \leq r$ il round in cui i ha inserito $\hat{b}$ in $\mathcal{E}_i$.

Per farlo, nel round $t-1$ il nodo i deve aver ricevuto un messaggio $\langle \hat{b} \rangle_{1,\sigma_1,\ldots,\sigma_{t-1}}$ contenente t firme valide e distinte, inclusa quella della sorgente.

Poiché i è onesto, nel round t esso firma il messaggio e invia a tutti i nodi $\langle \hat{b} \rangle_{1,\sigma_1,\ldots,\sigma_{t-1},\sigma_i}$.

Quindi, entro la fine del round t, ogni nodo onesto j riceve un messaggio contenente $\hat{b}$ con $t+1$ firme valide e distinte.

Ne segue che, alla fine del round $t+1$, ogni nodo onesto j ha necessariamente $\hat{b} \in \mathcal{E}_j$: o perché lo aveva già inserito in un round precedente, oppure perché lo inserisce proprio nel round $t+1$.

Poiché $t \leq r$, otteniamo in particolare che ogni nodo onesto j ha $\hat{b} \in \mathcal{E}_j$ alla fine del round $r+1$. □

Lemma 2. *Se un nodo onesto i ha $\hat{b} \in \mathcal{E}_i$ alla fine del round $f+1$, allora ogni nodo onesto j ha $\hat{b} \in \mathcal{E}_j$ alla fine del round $f+1$.*

dim. Lemma 2. Sia i un nodo onesto tale che $\hat{b} \in \mathcal{E}_i$ alla fine del round $f+1$, e sia $t \leq f+1$ il round in cui i ha inserito $\hat{b}$ in $\mathcal{E}_i$.

Distinguiamo due casi.

- **caso 1:** $t \leq f$. In questo caso, applicando il Lemma 1 con $r = t$, otteniamo che ogni nodo onesto j ha $\hat{b} \in \mathcal{E}_j$ alla fine del round $t+1$. Siccome $t+1 \leq f+1$, ne segue che ogni nodo onesto j ha ancora $\hat{b} \in \mathcal{E}_j$ alla fine del round $f+1$.

- **caso 2:** $t = f+1$.

  In questo caso il nodo i ha inserito $\hat{b}$ in $\mathcal{E}_i$ al round finale, quindi nel round f ha ricevuto un messaggio contenente $\hat{b}$ con $f+1$ firme valide e distinte, inclusa quella della sorgente.

Poiché al più f nodi sono corrotti, tra queste $f+1$ firme ce n'è almeno una appartenente a un nodo onesto, diciamo j.

Affinché il nodo onesto j abbia firmato un messaggio contenente $\hat{b}$, esso deve aver inserito $\hat{b}$ in $\mathcal{E}_j$ in un round precedente $t' \leq f$.

Applicando allora il Lemma 1 al nodo j e al round t', concludiamo che ogni nodo onesto ha $\hat{b} \in \mathcal{E}$ entro il round $t'+1 \leq f+1$. In particolare, ogni nodo onesto j' ha $\hat{b} \in \mathcal{E}_{j'}$ alla fine del round $f+1$.

In entrambi i casi, ogni nodo onesto ha $\hat{b} \in \mathcal{E}$ alla fine del round $f+1$. □

□

5 Schemi di firme digitali

Uno schema di firma digitale è una tripla di algoritmi (probabilistici) polinomiali $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$:

- $\mathsf{KeyGen}(1^\lambda) = (sk, pk)$
  Prende in input un security parameter $\lambda \in \mathbb{N}$ (in unario 1^λ) e restituisce una coppia di chiavi $sk \in \{0,1\}^{p(\lambda)}$ e $pk \in \{0,1\}^{p(\lambda)}$, dove p è un polinomio.

- $\mathsf{Sign}(sk, m) = \sigma$
  Prende in input una chiave segreta sk e un messaggio $m \in \{0,1\}^*$ e restituisce una firma $\sigma \in \{0,1\}^{p(\lambda)}$.

- $\mathsf{Verify}(m, pk, \sigma) = \mathsf{True}/\mathsf{False}$
  Prende in input un messaggio $m \in \{0,1\}^*$, una chiave pubblica pk e una firma σ e restituisce True o False. Diciamo che σ è una firma valida per m rispetto alla chiave pubblica pk se $\mathsf{Verify}(m, pk, \sigma) = \mathsf{True}$.

Lo schema è corretto se per ogni $\lambda \in \mathbb{N}$, per ogni coppia di chiavi (sk, pk) generata con $\mathsf{KeyGen}(1^\lambda)$ e per ogni messaggio $m \in \{0,1\}^*$ risulta che

$$\mathsf{Verify}(m, pk, \mathsf{Sign}(sk, m)) = \mathsf{True}.$$

Diciamo che lo schema è sicuro se, data una coppia di chiavi (sk, pk) generata con $\mathsf{KeyGen}(1^\lambda)$, dato un qualunque algoritmo (probabilistico) polinomiale A che

1. Prende in input pk;
2. Può fare chiamate alla funzione $\mathsf{Sign}(sk, \cdot)$ (e quindi ottenere un numero polinomiale di coppie $(\hat{m}, \hat{\sigma})$, dove $\hat{\sigma}$ è una firma valida per $\hat{m}$ rispetto a pk);
3. Restituisce in output una coppia (m, σ);

allora per la coppia (m, σ) data in output dall'algoritmo deve valere che, se m non è uno dei messaggi su cui l'algoritmo ha chiamato la funzione $\mathsf{Sign}(sk, \cdot)$, la probabilità che σ sia una firma valida per m rispetto alla chiave pubblica pk è *negligible* in λ (una funzione $f(\lambda)$ è *negligible* se va a zero più velocemente di qualunque polinomio al crescere di λ).

5.1 Esempio: RSA

Nello schema di firma digitale RSA:

- $\mathsf{KeyGen}(1^\lambda)$
  - Genera due numeri primi p e q dell'ordine di λ bit;

  - Calcola $n = pq$;
  - Genera un e coprimo con $(p-1)(q-1)$;
  - Calcola l'inversa di e modulo $(p-1)(q-1)$, ossia un d tale che

$$de \equiv 1 \pmod{(p-1)(q-1)};$$

  - Restituisce $pk = (n, e)$, $sk = (n, d)$.

- $\mathsf{Sign}(sk, m)$
  - Calcola $h = \mathsf{Hash}(m)$, dove Hash è una funzione hash crittografica;
  - Restituisce $h^d \bmod n$.

- $\mathsf{Verify}(m, pk, \sigma)$
  - Calcola $h = \mathsf{Hash}(m)$;
  - Se $\sigma^e \equiv h \pmod{n}$ restituisce True, altrimenti restituisce False.

Nel modello sincrono con PKI setup, il protocollo di Dolev–Strong risolve il problema Byzantine Broadcast (BB) qualunque sia il numero f di nodi corrotti, in $f+1$ round di comunicazione. Se rimuoviamo l'ipotesi che ci sia un PKI setup, allora nello stesso modello nessun protocollo può garantire validity e consistency contemporaneamente, se ci sono $n/3$ o più nodi corrotti, dove n è il numero totale di nodi.

6 Lower bound per Byzantine Broadcast senza PKI

Teorema 1. *In assenza di PKI setup e con $f \geq n/3$ nodi corrotti, nessun protocollo per il problema Byzantine Broadcast può garantire validity e consistency.*

Dimostrazione. Consideriamo il sistema G in Figura 1 con 3 nodi, dove il nodo A è la sorgente, e supponiamo per assurdo che esista un protocollo Π che garantisce validity e consistency nel sistema G anche in presenza di un nodo corrotto.

Consideriamo poi l'esperimento concettuale H in Figura 1 dove gli archi del grafo indicano i canali di comunicazione (quindi ogni nodo può comunicare soltanto con i suoi due vicini) e:

- x e x' sono copie esatte di A (quindi, in particolare, entrambi i nodi "credono" di essere la sorgente A del problema BB);
- y e y' sono copie esatte di B;
- z e z' sono copie esatte di C.

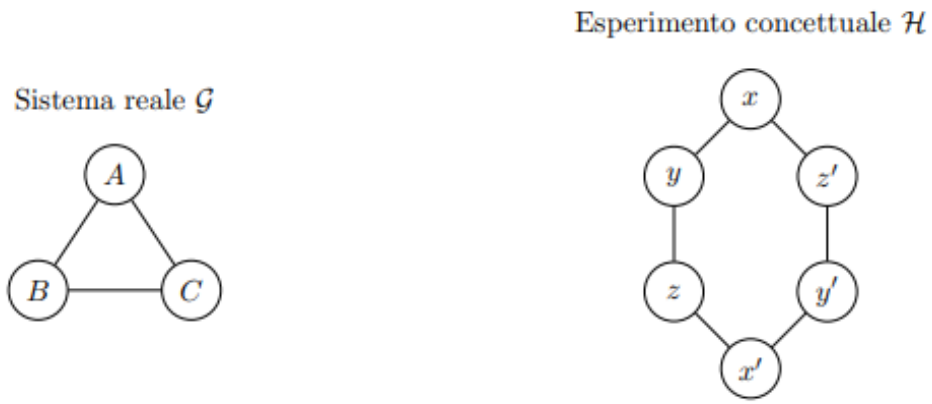


Figura 1: Il sistema reale $\mathcal{G}$ e l'esperimento concettuale $\mathcal{H}$

Osserviamo che un nodo non può distinguere se si trova nel sistema reale G o nell'esperimento concettuale H: per esempio, il nodo z' in H è una copia del nodo C e comunica con due nodi, x e y', il primo è una copia del nodo A e il secondo è una copia del nodo B. L'esecuzione del protocollo Π può essere quindi "simulata" nell'esperimento concettuale H (i nodi x e x' eseguono le istruzioni previste per A, y e y' quelle previste per B e z e z' quelle previste per C). Ci chiediamo cosa danno in output i sei nodi in H se simuliamo l'esecuzione del protocollo Π dove al nodo x (una delle due copie della sorgente A) viene dato in input il bit 1 e al nodo x' (l'altra copia della sorgente A) viene dato in input 0.

Claim 1. *I nodi x e y devono dare in output* 1.

dim. Claim 1 Se eseguiamo il protocollo Π in G dando ad A input 1 e se A e B sono nodi onesti mentre C è corrotto, allora A e B devono dare in output 1 qualunque cosa faccia C, perché stiamo assumendo per ipotesi che Π soddisfi validity anche in presenza di un nodo corrotto. Ora osserviamo che il comportamento del nodo corrotto C potrebbe essere il seguente: per decidere quali messaggi inviare ad A e B il nodo C potrebbe

- Simulare l'esecuzione del protocollo Π in H;
- Inviare a B tutti i messaggi che nella simulazione il nodo z (che è una copia di C) invia ad y (che è una copia di B);
- Inviare ad A tutti i messaggi che nella simulazione il nodo z' (che è una copia di C) invia ad x (che è una copia di A).

I nodi x e y (che sono copie di A e B) allora devono dare in output esattamente quello che danno in output i due nodi onesti A e B in G, ossia 1 perché ricevono in H gli stessi messaggi che ricevono A e B in G.

Claim 2. *I nodi x' e z devono dare in output* 0.

dim. Claim 2

Claim 3. *I nodi y e z devono dare in output lo stesso bit.*

dim. Claim 3 Siccome i Claim 1 2 e 3 non possono essere tutti e tre veri abbiamo trovato un assurdo. Quindi la nostra ipotesi di partenza, ossia che il protocollo gratansice *validity* e *consistency* nel sistema G in presenza di un nodo corrotto, non può essere vera.

Abbiamo visto che senza PKI setup nessun protocollo per Byzantine Broadcast può garantire contemporaneamente validity e consistency se ci sono $f \geq n/3$ nodi corrotti. Vediamo un semplice protocollo probabilistico che, se il numero di nodi corrotti è $f < n/3$, garantisce validity con probabilità 1 e soddisfa consistency con alta probabilità (w.h.p.)[1].

Il protocollo fa uso di una funzione H che nella dimostrazione assumiamo essere un random oracle. In una implementazione pratica, al posto di un random oracle si usa una funzione hash crittografica.

7 Un protocollo probabilistico per BB

Sia $H : \mathbb{N} \cup \{0\} \to [n]$ una funzione tale che $H(0) = 1$ e per ogni $t \in \mathbb{N}$ si comporta come un random oracle. Consideriamo il protocollo seguente per il problema Byzantine Broadcast.

Algorithm 2 Synchronous Randomized Byzantine Broadcast

1: ROUND 0. Ogni nodo u inizializza una variabile $\mathsf{sb}_u = \perp$.
La sorgente (il nodo 1) riceve in input b e inizializza $\mathsf{sb}_1 = b$.
2: PER OGNI ITERAZIONE $t = 0, \ldots, k-1$:
3: ROUND $3t$. Si seleziona il *leader* ℓ_t dell'iterazione utilizzando il random oracle $\ell_t = H(t)$
4: Se $\mathsf{sb}_{\ell_t} \neq \perp$ allora ℓ_t invia a tutti sb_{ℓ_t};
5: Altrimenti ℓ_t sceglie un bit $\{0, 1\}$ u.a.r. e lo invia a tutti.
6: ROUND $3t + 1$. Ogni nodo u:
7: Se $\mathsf{sb}_u \neq \perp$ allora u invia a tutti sb_u;
8: Altrimenti u invia a tutti il bit ricevuto nel round $3t$ dal leader ℓ_t (se non ha ricevuto nulla da ℓ_t o ha ricevuto entrambi i bit, invia 0 o 1 arbitrariamente)
9: ROUND $3t + 2$. Ogni nodo u:
10: Se c'è un bit $\hat{b}$ che u ha ricevuto nel round $3t + 1$ da almeno $2n/3$ nodi distinti allora u imposta $\mathsf{sb}_u = \hat{b}$;
11: Altrimenti imposta $\mathsf{sb}_u = \perp$.
12: ROUND $3k$. Ogni nodo u:
13: OUTPUT sb_u

Teorema 2. *Se il numero di nodi corrotti è $f < n/3$ allora il protocollo in Algorithm 2 soddisfa Validity con probabilità 1 e Consistency con probabilità almeno $1 - \left(\frac{2}{3}\right)^k$.*

Sketch of Proof. Per Validity osservare che, se la sorgente è onesta, nell'iterazione $t = 0$ succede questo: nel Round 0 tutti i nodi onesti ricevono dalla sorgente il bit b (linea 4); nel Round 1 ogni nodo onesto invia b a tutti (linea 8); nel Round 2 ogni nodo onesto u riceve b da almeno $2n/3$ nodi distinti e quindi fissa il suo $sb_u = b$. Nelle iterazioni successive $t = 1, \ldots, k-1$ ogni nodo onesto u continua a inviare $sb_u = b$ (linea 7) e quindi il valore di sb_u rimane b (linea 10).

Per Consistency la dimostrazione procede come segue:

1. Si dimostra che in ogni iterazione t non possono esserci due nodi onesti u e v tali che alla fine dell'iterazione t abbiano $sb_u = 0$ e $sb_v = 1$;

   dim. Supponiamo per assurdo che esistano due nodi onesti u e v tali che, alla fine dell'iterazione t, si abbia $sb_u = 0$ e $sb_v = 1$.. Siano:

$$S_u = \{\, nodo \in [n] : u \text{ ha ricevuto il bit 0 in quell'iterazione} \,\},$$

[1] Data una successione di eventi $\{E_n\}_n$ diciamo che E_n avviene con alta probabilità (e scriveremo w.h.p. per *with high probability*) se esistono un $n_0 \in \mathbb{N}$ e un $c > 0$ tali che $\mathbb{P}(E_n) \geq 1 - n^{-c}$ per ogni $n \geq n_0$. Nel caso in questione l'evento E_n è: "In un sistema con n nodi il protocollo soddisfa consistency".

$$S_v = \{\, nodo \in [n] : v \text{ ha ricevuto il bit 1 in quell'iterazione} \,\},$$

quindi

$$|S_u| \geq \frac{2n}{3}, \qquad |S_v| \geq \frac{2n}{3}.$$

Osserviamo ora che

$$|S_u \cap S_v| = |S_u| + |S_v| - |S_u \cup S_v| \geq \frac{2n}{3} + \frac{2n}{3} - n = \frac{n}{3},$$

(poiché chiaramente $|S_u \cup S_v| \leq n$.)

Dunque esistono almeno $n/3$ nodi che appartengono a $S_u \cap S_v$. Ciascuno di questi nodi ha inviato il bit 0 a u e il bit 1 a v nello stesso round, quindi deve essere un nodo corrotto.

Ne segue che il numero di nodi corrotti è almeno

$$f \geq \frac{n}{3},$$

in contraddizione con l'ipotesi $f < n/3$. □

2. Si definisce una iterazione t *lucky* se si verifica che:
   (i) il leader ℓ_t è onesto;
   (ii) il bit inviato dal leader (linee 4–5) è lo stesso posseduto da ogni altro nodo onesto u che abbia $sb_u \neq \perp$ all'inizio dell'iterazione t;
3. Si osserva che, se esiste una iterazione lucky, allora tutti i nodi onesti danno in output lo stesso bit;
4. Si dimostra che, qualunque cosa sia successa nelle iterazioni precedenti, una iterazione t è lucky con probabilità almeno 1/3;
5. Si osserva che la probabilità che fra le k iterazioni nemmeno una sia lucky è quindi minore di $(2/3)^k$.

dim. Per ogni iterazione t, definiamo l'evento $\mathcal{E}_t = \{$"l'iterazione t è lucky"$\}$.

E sappiamo che, $\Pr[\mathcal{E}_t] \geq \frac{1}{3}$.

Quindi $\Pr[\overline{\mathcal{E}_t}] \leq \frac{2}{3}$.

La probabilità che nessuna delle k iterazioni sia lucky è dunque al più

$$\Pr\left[\bigcap_{t=1}^{k} \overline{\mathcal{E}_t}\right] \leq \left(\frac{2}{3}\right)^k.$$

Pertanto, la probabilità che esista almeno un'iterazione lucky tra le k iterazioni è almeno $1 - \left(\frac{2}{3}\right)^k$. □

Scelta di k. Se vogliamo che

$$1 - \varepsilon^k \geq 1 - \frac{1}{n^c},$$

allora deve valere

$$\varepsilon^k \leq \frac{1}{n^c}.$$

Elevando entrambi i membri otteniamo

$$e^{k \log(\varepsilon)} \leq e^{\log(\frac{1}{n^c})},$$

cioè

$$k \log \varepsilon \leq \log(\frac{1}{n^c}) \qquad = \qquad k \log \frac{1}{\varepsilon} \geq \log(n^c)$$

Quindi

$$k \geq \frac{c \log n}{\log(1/\varepsilon)}.$$

Nel nostro caso $\varepsilon = \frac{2}{3}$, dunque

$$k \geq \frac{c \log n}{\log(3/2)}.$$

Introduzione del *Byzantine Agreement* (BA), e il modello asincrono, nel quale non assumiamo che ci sia un clock globale né che i messaggi arrivino a destinazione nello stesso ordine nel quale sono partiti.

8 Byzantine Agreement

Nel problema *Byzantine Agreement* (BA) abbiamo n nodi di cui f sono corrotti. A ogni nodo i viene affidato in input un messaggio b_i. Vogliamo progettare un protocollo, che termini in un tempo finito, in cui ogni nodo onesto i dia in output un valore y_i tale che:

- **Consistency:** se i e j sono due nodi onesti, allora $y_i = y_j$;
- **Validity:** se tutti i nodi onesti hanno in input lo stesso bit b, allora $y_i = b$ per ogni nodo onesto i.

9 Modello asincrono

Nel modello asincrono non abbiamo un *global clock* e non abbiamo un *bound* sul tempo che impiega un messaggio inviato da un nodo a raggiungere il nodo di destinazione.

Un *protocollo* nel modello asincrono è *event driven*, cioè specifica cosa fa un nodo nel momento in cui riceve un messaggio: il nodo può eseguire un numero arbitrario di operazioni locali, cambiare stato (lo stato di un nodo è costituito dai valori delle sue variabili locali, che possono dipendere dall'input e da tutti i messaggi ricevuti). Una configurazione C è una coppia $C = (S, M)$ dove S rappresenta lo stato di ogni nodo e M è la *message pool*, ossia l'insieme di messaggi non ancora consegnati. Un messaggio m è una coppia $m = (p, x)$ dove p è il nodo destinatario del messaggio e x è il contenuto del messaggio.

Se Π è un protocollo deterministico nel modello asincrono, data una configurazione $C = (S, M)$ e un messaggio $m = (p, x) \in M$, indichiamo con $m(C)$ la configurazione $C' = (S', M')$ che si ottiene in base al protocollo quando il messaggio m viene consegnato, ossia l'insieme S' è quello che si ottiene da S aggiornando lo stato del nodo p dopo l'elaborazione delle informazioni contenute in x, e M' è la *message pool* che si ottiene da M togliendo il messaggio appena consegnato m e aggiungendo tutti i messaggi inviati da p a seguito della ricezione del messaggio m.

Uno *schedule* σ è una sequenza ordinata di messaggi $\sigma = \big((p_1, x_1), \ldots, (p_k, x_k)\big)$. Diciamo che uno schedule σ è applicabile a una configurazione C se $m_1 \in M$ e se per ogni $i = 2, \ldots, k$, il messaggio m_i appartiene alla message pool della configurazione $\sigma_{[1:i-1]}(C)$, dove con $\sigma_{[1:i-1]}$ intendiamo lo schedule formato dai primi $i - 1$ messaggi dello schedule σ.

Dato un protocollo Π, una configurazione C e uno schedule σ, indichiamo con $\sigma(C)$ la configurazione che si ottiene partendo da C in base al protocollo Π dopo la consegna di tutti i messaggi in σ (nell'ordine stabilito dalla sequenza).

Oss. Dato un protocollo deterministico Π, una configurazione $C = (S, M)$ e due schedule $\sigma_1 = \big((p_1, x_1), \ldots, (p_k, x_k)\big)$ e $\sigma_2 = \big((q_1, y_1), \ldots, (q_h, y_h)\big)$ applicabili a C, se gli insiemi di nodi destinatari nei due schedule sono disgiunti, $\{p_1, \ldots, p_k\} \cap \{q_1, \ldots, q_h\} = \emptyset$, allora $\sigma_2(\sigma_1(C)) = \sigma_1(\sigma_2(C))$.

Teorema 3 (Fischer, Lynch, Paterson)**.** *Nel modello asincrono nessun protocollo deterministico per Byzantine Agreement che termina in un tempo finito può soddisfare consistency e validity, se c'è almeno un nodo corrotto.*

Sketch of Proof. La dimostrazione procede mostrando che se Π è un protocollo deterministico per Byzantine Agreement che soddisfa validity e consistency, allora esiste una strategia dell'avversario che fa in modo che il protocollo non termini mai.

Se Π_{BA} è un protocollo per Byzantine Agreement che soddisfa validity e consistency, diciamo che una configurazione C è *0-valente* (rispettivamente *1-valente*) per il protocollo Π se, qualunque cosa faccia l'avversario, a partire dalla configurazione C il protocollo fa in modo che tutti i nodi onesti diano in output 0 (rispettivamente 1). Altrimenti diciamo che una configurazione C è *bivalente*.

Informalmente, la dimostrazione procede come segue:

1. Si dimostra che esiste un'assegnazione degli input ai nodi tale che la configurazione iniziale C_0 è bivalente;

2. Si dimostra che se C è una configurazione bivalente, allora l'avversario può fare in modo che, partendo da C, il protocollo raggiunga una nuova configurazione bivalente C'.

Dai punti 1 e 2 segue che l'avversario può fare in modo che il protocollo Π_{BA} non termini mai.

Dimostriamo il punto 1 nel lemma seguente. Vedremo la dimostrazione del punto 2 nella prossima lezione.

Lemma 3. *Se Π_{BA} è un protocollo deterministico per Byzantine Agreement che termina e soddisfa validity e consistency, allora esiste un'assegnazione degli input ai nodi tale che la configurazione iniziale è bivalente.*

dim. Lemma 3. Per $i = 0, 1, \ldots, n$, sia X_i la configurazione iniziale in cui gli input dei nodi minori o uguali a i sono 1 e gli input dei nodi maggiori di i sono 0.

Siccome per ipotesi Π_{BA} soddisfa validity, la configurazione X_0 (in cui tutti gli input sono 0) deve essere 0-valente e la configurazione X_n (in cui tutti gli input sono 1) deve essere 1-valente.

Sia i il più piccolo indice tale che X_{i-1} è 0-valente e X_i non è 0-valente. Mostriamo che X_i non può essere 1-valente, e quindi deve essere bivalente.

Siccome per ipotesi Π_{BA} termina anche in presenza di un nodo corrotto, deve esistere uno schedule σ applicabile a X_{i-1} tale che σ non contiene messaggi per il nodo i e $\sigma(X_{i-1})$ è una configurazione in cui tutti i nodi diversi da i che seguono il protocollo hanno deciso il loro output. Infatti, se un tale schedule non ci fosse, nel caso in cui i fosse un nodo corrotto che non fa nulla ogni volta che riceve un messaggio, non ci sarebbe nessuno schedule che consenta agli altri $n-1$ nodi di terminare.

Siccome X_{i-1} è una configurazione 0-valente, nella configurazione $\sigma(X_{i-1})$ tutti i nodi diversi da i devono dare in output 0. Siccome le configurazioni X_{i-1} e X_i differiscono solo per l'input del nodo i e lo schedule σ non contiene messaggi per il nodo i, anche nella configurazione $\sigma(X_i)$ tutti i nodi diversi da i devono aver deciso che il loro output è 0.

Quindi la configurazione iniziale X_i non può essere 1-valente e, siccome non è neanche 0-valente per costruzione, deve essere bivalente. $\square$

VEDERE APPUNTI LEZIONE 5 E VEDERE APPUNTI LEZIONE 3 CON DIMOSTRAZIONE CLAIM

Si completa la dimostrazione del seguente teorema.

Teorema 4 (Fischer, Lynch, Paterson)**.** *Nel modello asincrono nessun protocollo deterministico per Byzantine Agreement che termina in un tempo finito può soddisfare consistency e validity, se c'è almeno un nodo corrotto.*

10 Dimostrazione del teorema FLP

Si ricordi che, dato un protocollo deterministico Π per Byzantine Agreement, una configurazione $C = (S, M)$ e uno schedule $\sigma = (m_1, \ldots, m_k)$, diciamo che σ è applicabile a C se $m_1 \in M$ e se per ogni $i = 2, \ldots, k$, il messaggio m_i appartiene alla message pool della configurazione $\sigma[1 : i-1](C)$, dove con $\sigma[1 : i-1]$ intendiamo lo schedule formato dai primi $i-1$ messaggi dello schedule σ. Si ricordi inoltre che una configurazione C si dice *0-valente* (rispettivamente *1-valente*) per il protocollo Π se, qualunque cosa faccia l'avversario, a partire dalla configurazione C il protocollo fa in modo che tutti i nodi onesti diano in output 0 (rispettivamente 1). Altrimenti diciamo che una configurazione C è *bivalente*.

Lemma 4. *Se Π_{BA} è un protocollo deterministico per Byzantine Agreement che termina e soddisfa validity e consistency, allora esiste un'assegnazione degli input ai nodi tale che la configurazione iniziale è bivalente.*

Nel seguito di queste note diamo una dimostrazione del secondo lemma chiave.

Lemma 5. *Se Π_{BA} è un protocollo deterministico per Byzantine Agreement che termina e soddisfa validity e consistency, $C = (S, M)$ è una configurazione bivalente per Π_{BA} e $m = (p, x)$ è un messaggio in M, allora esiste uno schedule $\sigma = (m_1 = (p_1, x_1), \ldots, m_k = (p_k, x_k))$ applicabile a C tale che:*

1. *$m_k = m$;*
2. *$\sigma(C)$ è una configurazione bivalente per Π_{BA}.*

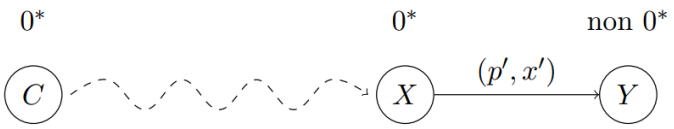
dim. Lemma 5. Se $m(C)$ è bivalente, il lemma è dimostrato. Altrimenti $m(C)$ è 0-valente o 1-valente. Supponiamo senza perdita di generalità che $m(C)$ sia 0-valente; il caso in cui $m(C)$ è 1-valente è speculare.

Data una configurazione bivalente $\hat{C}$ per Π_{BA}, diciamo che $\hat{C}$ è di tipo 0^* per il nostro messaggio m se $m(\hat{C})$ è 0-valente. Analogamente diciamo che $\hat{C}$ è di tipo 1^* o bi*, se $m(\hat{C})$ è 1-valente o bivalente, rispettivamente.

Siccome ci siamo posti, senza perdita di generalità, nel caso in cui $m(C)$ è 0-valente, con la terminologia appena definita possiamo dire che C è una configurazione bivalente di tipo 0^*.

Consideriamo tutte le configurazioni raggiungibili applicando a C degli schedule che non contengono il nostro messaggio $m = (p, x)$. Osserviamo che fra queste configurazioni ce ne deve essere qualcuna che non è di tipo 0^*, perché se fossero tutte di tipo 0^* allora la configurazione C non potrebbe che essere 0-valente, mentre è bivalente per ipotesi.

Indichiamo con Y la prima configurazione non 0^* raggiungibile applicando a C uno schedule che non contiene il messaggio m, indichiamo con X la configurazione di tipo 0^* che la precede e con $m' = (p', x')$ il messaggio dello schedule che secondo il protocollo porta dalla configurazione X alla configurazione Y.



Il nostro messaggio $m = (p, x)$ è applicabile alla configurazione Y, perché stava nella message pool della configurazione C e non è mai stato consegnato, quindi sta anche nella message pool di Y. Sia allora $Z = m(Y)$ la configurazione che si ottiene applicando m a Y. Mostreremo che Z è la configurazione bivalente che stiamo cercando, concludendo così la dimostrazione del lemma.

La configurazione Z non può essere 0-valente, altrimenti Y sarebbe di tipo 0^*. Ora mostriamo che Z non può essere neanche 1-valente, esibendo uno schedule applicabile a Z che conduce a una configurazione in cui tutti i nodi onesti danno in output 0.

Prima di tutto osserviamo che se Z non è 0-valente allora deve essere $p = p'$. Infatti, se fosse $p \neq p'$, la configurazione Z, che si ottiene da X consegnando prima (p', x') e poi (p, x), sarebbe la stessa configurazione che si ottiene da X consegnando prima (p, x) e poi (p', x') (vedi l'Esercizio 4 degli appunti del 16 marzo), ma quest'ultima configurazione è sicuramente 0-valente, perché X è di tipo 0^*.

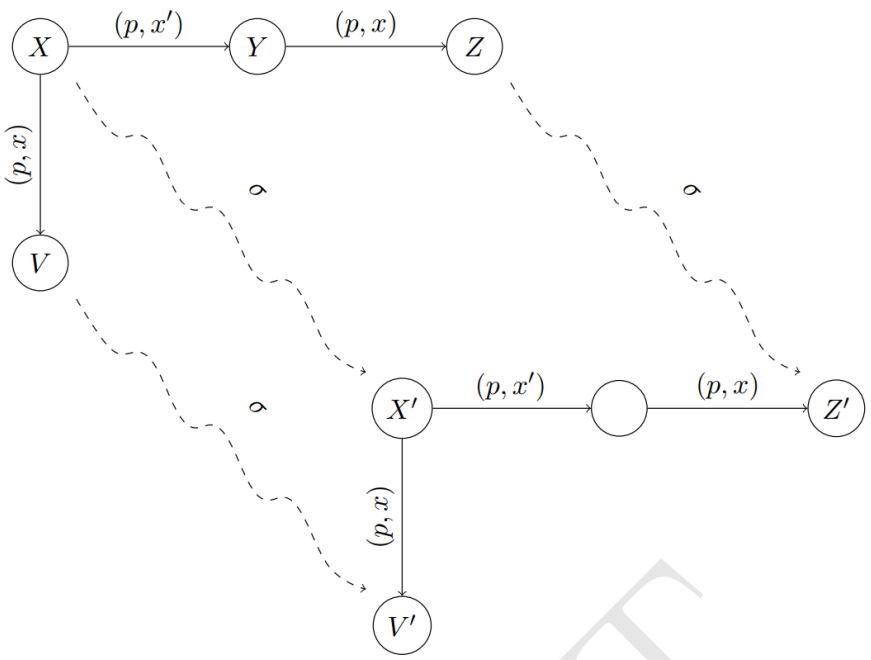
Siccome $p = p'$ e siccome per ipotesi Π_{BA} termina anche in presenza di un nodo corrotto, deve esistere uno schedule σ applicabile a X tale che σ non contiene messaggi per il nodo p e $X' := \sigma(X)$ è una configurazione in cui tutti i nodi diversi da p che seguono il protocollo hanno deciso il loro output. Infatti, se un tale schedule non ci fosse, nel caso in cui p è un nodo corrotto che non fa nulla ogni volta che riceve un messaggio, nessuno schedule consentirebbe agli altri $n - 1$ nodi di terminare.

Lo schedule σ non contiene messaggi per il nodo p, quindi è applicabile anche alla configurazione $m(X)$ e, poiché $p' = p$, anche alla configurazione Z.

1. Siccome $\sigma(X)$ è una configurazione in cui tutti i nodi diversi da p devono aver deciso il loro output, anche $m(\sigma(X))$ lo è. Ma quindi l'output deciso dai nodi deve essere 0, perché $m(\sigma(X)) = \sigma(m(X))$ e $m(X)$ è una configurazione 0-valente dato che X è di tipo 0^*.

2. La configurazione $\sigma(Z) = \sigma(m(m'(X))) = m(m'(\sigma(X)))$ e siccome $\sigma(X)$ è una configurazione in cui tutti i nodi diversi da p hanno deciso che il loro output è 0, anche $\sigma(Z)$ lo è.

Quindi abbiamo trovato uno schedule σ applicabile a Z tale che i nodi danno in output 0. Quindi Z non può essere neanche 1-valente e pertanto deve essere bivalente. $\square$



Usando il lemma precedente e il lemma sugli input iniziali, possiamo infine dare una dimostrazione del teorema.

dim. Teorema 4. Supponiamo per assurdo che esista un protocollo deterministico Π_{BA} che termina e soddisfa validity e consistency nel modello asincrono anche in presenza di un nodo corrotto.

Per il lemma precedente deve esistere una configurazione iniziale bivalente $C_0 = (S_0, M_0)$ per Π_{BA}. Per ogni $i \geq 1$, sia m_i il messaggio "più vecchio" presente in M_{i-1} (rompendo la simmetria arbitrariamente in caso di più messaggi), sia σ_i lo schedule che termina con m_i la cui esistenza è garantita dal lemma precedente e sia $C_i = \sigma_i(C_{i-1})$ la configurazione bivalente risultante.

Lo schedule $(\sigma_1, \sigma_2, \sigma_3, \dots)$ è quindi uno schedule infinito che garantisce che ogni messaggio inserito nella message pool viene prima o poi consegnato e fa in modo che il protocollo rimanga sempre in configurazioni bivalenti.

Questo contraddice l'ipotesi che il protocollo termini in un tempo finito. Quindi nel modello asincrono non può esistere nessun protocollo deterministico per Byzantine Agreement che termina in un tempo finito e soddisfa validity e consistency, se c'è almeno un nodo corrotto. □

Il Teorema 1 afferma che nel modello asincrono non può esistere nessun protocollo deterministico per Byzantine Agreement in presenza di qualche nodo corrotto. Questo non esclude la possibilità che sia possibile progettare protocolli probabilistici; infatti ce ne sono diversi e nella prossima lezione ne analizzeremo uno.